

Tutorial Swift: Introdução ao SwiftUI - Componentes Visuais e State

Objetivo: Criar uma interface de usuário básica com SwiftUI, explorando componentes visuais fundamentais e gerenciamento de estado.

SwiftUI é um framework de interface do usuário declarativo e moderno da Apple, projetado para simplificar o desenvolvimento de aplicativos em todas as plataformas da empresa (iOS, macOS, watchOS, tvOS). Ao invés de construir interfaces visualmente através de Storyboards ou XIBs, o SwiftUI permite descrever a interface com código Swift, combinando componentes pré-definidos (como Text, Image, Button) e modificadores (como padding, background) para criar layouts flexíveis e responsivos. O SwiftUI também facilita o gerenciamento de estado, garantindo que a interface se atualize automaticamente quando os dados subjacentes mudam, resultando em um fluxo de desenvolvimento mais intuitivo e eficiente.

Projeto: Contador Simples

O que faremos: Criaremos um aplicativo com um label para exibir um contador, e dois botões: um para incrementar e outro para decrementar o contador.

Passo 1: Criar um Novo Projeto SwiftUI

1. Abra o Xcode e selecione "Create a new Xcode project".
2. Escolha "App" na aba iOS e clique em "Next".
3. Configure o projeto:
 - * Product Name: "SwiftUIContador"
 - * Interface: **SwiftUI** (Crucial!)

* Language: Swift

Choose options for your new project:

Product Name: SwiftUICo

Team: Add account

Organization Identifier: br.pucpr.c

Bundle Identifier: br.pucpr.c

Interface: SwiftUI

Language: Swift

Testing System: None

Storage: None

☐ Host in

4. Clique em "Next" e escolha onde salvar o projeto.

Passo 2: Entendendo o Arquivo `ContentView.swift`

O Xcode criará um arquivo chamado `ContentView.swift`. Ele contém o código da interface principal do seu aplicativo. Abra-o. Você verá algo parecido com:

```
import SwiftUI

struct ContentView: View {
    var body: some View {
        VStack {
            Image(systemName: "globe")
                .imageScale(.large)
                .foregroundColor(.accentColor)
            Text("Hello, world!")
        }
        .padding()
    }
}

struct ContentView_Previews: PreviewProvider {
    static var previews: some View {
        ContentView()
    }
}
```

import SwiftUI: Importa o framework SwiftUI, que fornece os componentes e funcionalidades necessárias para construir interfaces.

struct ContentView: View: Define uma struct chamada **ContentView** que conforma ao protocolo **View**. Em SwiftUI, a interface é construída usando structs que representam views.

SwiftUI usa structs para views primariamente por conta de sua natureza de tipo de valor (value type), o que se alinha perfeitamente com o paradigma de programação declarativa que o SwiftUI adota. Structs, ao serem copiadas, criam instâncias independentes, evitando efeitos colaterais indesejados quando se manipula o estado da interface. Essa imutabilidade implícita ajuda a prever o comportamento da view e facilita o rastreamento de mudanças, tornando o código mais simples, seguro e eficiente. Além disso, structs são mais leves que classes (tipos de referência) e otimizadas para operações de cópia e comparação, contribuindo para o desempenho geral do SwiftUI.

var body: some View: Esta é a propriedade mais importante. Ela retorna a descrição da interface do usuário para o SwiftUI.

VStack: *Um container que organiza as views em uma coluna vertical.*

Em SwiftUI, containers são views que organizam e controlam o layout de outras views. Eles são fundamentais para criar interfaces complexas e adaptáveis. Ao invés de especificar posições e tamanhos absolutos, você usa containers para definir regras sobre como as views devem ser dispostas. Os containers mais comuns em SwiftUI são:

- VStack: Organiza as views verticalmente (uma embaixo da outra).
- HStack: Organiza as views horizontalmente (uma ao lado da outra).
- ZStack: Empilha as views umas sobre as outras (em profundidade, como camadas).
- Grid (iOS 16+): Permite criar layouts de grade, organizando as views em linhas e colunas.

Esses containers, combinados com modifiers e o sistema de Auto Layout do SwiftUI, permitem criar interfaces flexíveis que se adaptam a diferentes tamanhos de tela, orientações e até mesmo a diferentes idiomas e tamanhos de fonte.

Image: Um componente que exibe uma imagem (neste caso, um ícone do sistema).

Em SwiftUI, componentes são os blocos de construção fundamentais para criar interfaces de usuário, conformando ao protocolo View. São structs que descrevem um trecho da interface, definindo sua aparência (layout, cores, fontes) e comportamento (interação com o usuário). Componentes podem ser tanto primitivos, como Text, Image e Button, quanto compostos, combinando outros componentes para criar layouts mais complexos. A grande vantagem é que o SwiftUI gerencia a renderização e atualização desses componentes de forma eficiente, com base no estado, permitindo criar interfaces dinâmicas e responsivas de forma declarativa.

Text: Um componente que exibe texto.

.padding(): *Um modifier que adiciona preenchimento (espaçamento) ao redor da VStack.*

Em SwiftUI, modifiers são métodos que você aplica a uma View para modificar sua aparência ou comportamento, sem alterar sua essência fundamental. Pense neles como camadas de personalização que você adiciona sobre a View base. Eles são encadeados de forma fluente, retornando uma nova View com as modificações aplicadas, o que significa que a View original permanece inalterada. Essa abordagem imutável e declarativa é crucial para o SwiftUI, permitindo que o framework otimize a renderização e rastreie as mudanças de estado de forma eficiente. Modifiers podem controlar desde a cor de fundo, a fonte, o espaçamento e o alinhamento até a adição de gestos e efeitos de animação, oferecendo uma grande flexibilidade na criação de interfaces personalizadas.

ContentView_Previews: Outra struct que permite ver uma pré-visualização da interface no Xcode Canvas (mais sobre isso depois).

Passo 3: Adicionar o Contador

1. Adicione um `@State` property para armazenar o valor do contador. Coloque este código dentro da struct `ContentView`, acima do `var body`:

```
@State private var contador = 0
```

@State: Um property wrapper que permite que o SwiftUI observe as mudanças nesta propriedade e reconstrua a interface quando ela mudar. É crucial para tornar a interface dinâmica.

O `@State` é um property wrapper fundamental em SwiftUI que permite a uma View armazenar e gerenciar seu próprio estado interno de forma observável. Essencialmente, ele cria uma fonte de verdade privada para um valor, e automaticamente faz com que a View se re-renderize (recompute its body) sempre que esse valor muda. É importante notar que o `@State` é local à View em que é declarado; outras Views não têm acesso direto a ele. Pense no `@State` como um "gatilho" que diz ao SwiftUI: "Ei, se essa propriedade mudar, a interface precisa ser redesenhada para refletir a nova informação". Ele é o alicerce para criar interfaces dinâmicas e interativas em SwiftUI.

private: Restringe o acesso a esta variável a apenas dentro desta struct.

2. Substitua o `Text("Hello, world!")` existente por um `Text` que exiba o valor do contador:

```
Text("Contador: \(contador)")  
    .font(.title) // Modifica a fonte para um tamanho de título
```

Passo 4: Adicionar os Botões

1. Adicione dois `Buttons` dentro da `VStack`, um para incrementar e outro para decrementar o contador:

```
Button("Incrementar") {  
    contador += 1 // Ação do botão  
}  
Button("Decrementar") {  
    contador -= 1 // Ação do botão  
}
```

Button: Um componente que exibe um botão interativo.

"Incrementar" e "Decrementar": Os títulos dos botões.

`{ contador += 1 }` e `{ contador -= 1 }`: Closures que são executadas quando os botões são tocados. Elas modificam o valor da propriedade `@State contador`.

O código completo da **ContentView** deve ficar assim:

```
import SwiftUI

struct ContentView: View {
    @State private var contador = 0

    var body: some View {
        VStack {
            Text("Contador: \(contador)")
                .font(.title)

            Button("Incrementar") {
                contador += 1
            }
            Button("Decrementar") {
                contador -= 1
            }
        }
        .padding()
    }
}

#Preview {
    ContentView()
}
```

Passo 5: Preview (Pré-visualização)

No Xcode, você deve ver a pré-visualização da sua interface no Canvas (geralmente à direita do editor de código). Se não estiver visível:

1. Certifique-se de que o arquivo **ContentView.swift** esteja aberto.
2. Vá em "Editor" -> "Canvas" para exibir o Canvas.
3. Se a pré-visualização não aparecer automaticamente, clique em "Resume" no topo do Canvas.

Você verá o texto "Contador: 0" e os dois botões. Toque nos botões na pré-visualização (ou execute o aplicativo no simulador) e você verá o contador sendo atualizado!

Passo 6: Melhorando o Layout

1. Adicione alguns modificadores para melhorar o layout:

```

VStack {
    Text("Contador: \$(contador)")
        .font(.title)
        .padding() // Adiciona espaço ao redor do texto

    HStack { // Organiza os botões horizontalmente
        Button("Incrementar") {
            contador += 1
        }
        .padding() // Espaçamento interno do botão
        .background(.blue) // Cor de fundo
        .foregroundColor(.white) // Cor do texto
        .cornerRadius(10) // Arredonda os cantos

        Button("Decrementar") {
            contador -= 1
        }
        .padding()
        .background(.red)
        .foregroundColor(.white)
        .cornerRadius(10)
    }
}
.padding() // Espaçamento externo da VStack

```

`HStack`: Organiza as views em uma linha horizontal.

.padding(): Adiciona espaço interno aos elementos.

.background(): Define a cor de fundo.

.foregroundColor(): Define a cor do texto.

.cornerRadius(): Arredonda os cantos.

Código Final:

```

import SwiftUI

struct ContentView: View {
    @State private var contador = 0

    var body: some View {
        VStack {
            Text("Contador: \$(contador)")
                .font(.title)
                .padding()

            HStack {

```



```

        Button("Incrementar") {
            contador += 1
        }
        .padding()
        .background(.blue)
        .foregroundColor(.white)
        .cornerRadius(10)

        Button("Decrementar") {
            contador -= 1
        }
        .padding()
        .background(.red)
        .foregroundColor(.white)
        .cornerRadius(10)
    }
    .padding()
}

#Preview {
    ContentView()
}

```

Parabéns! Você criou um aplicativo de contador simples com SwiftUI, aprendendo sobre **View**, **VStack**, **Text**, **Button**, modifiers e **@State**. Este é um ótimo ponto de partida para explorar recursos mais avançados do SwiftUI!

Exercício SwiftUI: Calculadora Básica

Objetivo: Criar uma calculadora simples em SwiftUI que permita aos usuários inserir dois números e realizar as operações de adição, subtração, multiplicação e divisão.

Requisitos:

1. Interface:

- * Dois **TextField**s para entrada dos números (operandos).
Quatro `Button`s para as operações: "+", "-", "x", "/".
- * Um **Text** para exibir o resultado.

2. Estado:

- * Use **@State** para armazenar os valores dos dois números (como **Double?**, pois podem ser vazios) e o resultado da operação (também como **Double?**).

3. Lógica:

- * Implemente as funções para realizar as operações:
 - * `somar(num1: Double?, num2: Double?) -> Double?`
 - * `subtrair(num1: Double?, num2: Double?) -> Double?`
 - * `multiplicar(num1: Double?, num2: Double?) -> Double?`
 - * `dividir(num1: Double?, num2: Double?) -> Double?`
 - * Lide com a possibilidade de os campos estarem vazios (`nil`) e com a divisão por zero (retornando `nil` nesses casos).
- * Quando um botão de operação for tocado, obtenha os valores dos `TextFields`, converta-os para `Double`, chame a função de operação correspondente e armazene o resultado na propriedade `@State` do resultado.
- * Exiba o resultado no `Text` da interface. Se o resultado for `nil`, exiba uma mensagem de erro (ex: "Entrada inválida" ou "Divisão por zero").

4. Layout:

- * Use `VStack` e `HStack` para organizar os elementos da interface de forma clara.
- * Adicione padding e outros modificadores para melhorar a aparência.

Extensões (Opcionais):

- * Adicione tratamento de erros mais robusto (ex: exibindo alertas).
- * Permita que o usuário limpe os campos de entrada.
- * Adicione mais operações (ex: potenciação, raiz quadrada).
- * Melhore a formatação do resultado (ex: limitando o número de casas decimais).

Dicas:

- * Use `.keyboardType(.numberPad)` nos `TextFields` para facilitar a entrada de números.
- * Lembre-se de que a conversão de `String` para `Double` pode falhar (retornando `nil`), então use optional binding (`if let` ou `guard let`) para lidar com essa possibilidade.
- * Use `NumberFormatter` para formatar a saída do resultado (ex: para limitar o número de casas decimais).